Overview

This project presents 4 implementations of the same software, each built using a different architecture. These architectures explore various approaches to software design, such as monolithic and microservices. Each implementation demonstrates how different architectures can impact the internal workings of the software.

Despite the different internal structures, the user interface and the available operations remain identical across all versions. The user will interact with the same software, transparently, regardless of the underlying architecture.

Key Highlights:

- `4 implementations` : Each version uses a different software architecture (clean, layer, modular and microservices).
- `Same UI` : The user interacts with the same interface and has access to the same features, regardless of the architecture.
- `Implementation transparency` : The architectural differences are "invisible" to the end user.

Project Structure

```
software_architectures_project/
├── clean-monolith/           # Clean Monolithic architecture implementation
├── layered-monolith/         # Layered Monolithic architecture implementation
├── modular-monolith/         # Modular Monolithic architecture implementation
└── microservices/            # Microservices architecture implementation
```

1. `clean-monolith/`

   This folder contains the implementation of the software using a clean architecture. Clean Architecture is a design approach that prioritizes the independence of the core business logic from external systems like frameworks, UI, and databases. It achieves this by structuring the system into concentric layers, where the core logic is at the center, and dependencies always point inward.

2. `layered-monolith/`

   This folder contains the implementation of the software using a layered architecture. Layered Architecture organizes software into distinct layers, each responsible for specific tasks, such as presentation, business logic, and data access. The layers communicate only with adjacent layers, ensuring a clear separation of concerns and improving modularity and maintainability.

3. `modular-monolith/`

   This folder contains the implementation of the software using a modular architecture. Modular Architecture divides a system into discrete, interchangeable modules that encapsulate specific functionality. Each module has a well-defined interface and can be developed, tested, and maintained independently, promoting reusability and scalability within the application.

4. `microservices/`

   This folder contains the implementation of the software using a microservices architecture. The application is split into smaller, independently deployable services, each responsible for a specific piece of functionality.

Common API

Despite the differences in architecture, all implementations share the same structure for user-facing elements:

- User Interface: The UI is consistent across all implementations and is located in a shared public/ or assets/ directory (or similar) within each folder.
- Operations: The core operations and API calls remain the same, ensuring a consistent user experience.

Products API

- GET: /products
- POST: /products
- GET: /products/{productId}
- DELETE: /products/{productId}

Recommendations API

- POST: /products/{productId}/recommendations
- DELETE: /products/{productId}/recommendations/{recommendationId}

Reviews API

- POST: /products/{productId}/reviews
- DELETE: /products/{productId}/reviews/{reviewId}

How to Run the 3 Monolithic Applications

Prerequisites

- **Java 21**
- **Maven** (for managing dependencies and building the project)

Navigate to the directory of the application

```
cd clean-monolith
```

2. Build the application using Maven

```
mvn package
```

3. Run the application

```
mvn spring-boot:run
```

The application should now be running and accessible at http://localhost:8080.

How to Run the Microservices Application

Prerequisites

- **Java 21**
- **Maven** (for managing dependencies and building the project)
- **Docker** (for running the application using Docker containers)
- **Docker Compose** (for orchestrating multi-container Docker applications)

1. Navigate to the directory of the service

```
cd microservices
```

2. Build the application using Maven

```
mvn package
```

Perform these first two steps for all the services that make up the application (product, review, recommendation, registry and gateway)

3. Build and start all services using Docker Compose

```
docker compose up --build
```

Viewing Services in Eureka After starting the services, you can view their registration on Eureka Server by visiting the following URL in your browser: http://localhost:8761 .

Please wait for a few seconds for all services to register. Once the registration is complete, you should see Product Service, Review Service, Recommendation Service and Gateway listed on the Eureka dashboard.

The application should now be running and accessible at http://localhost:8080.

Integrating a Thymeleaf UI into a Spring Boot Project

To integrate **Thymeleaf** into the Spring Boot project, the following steps were executed:

1. **Added the Thymeleaf Dependency**
   The Thymeleaf dependency was included in the `pom.xml` file:

   ```
   <dependency>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-thymeleaf</artifactId>
   </dependency>
   ```

2. **Configured Thymeleaf (Optional)**

   Thymeleaf was automatically configured by Spring Boot with default settings. However, additional customization was made in the application.properties file:

   ```
   spring.thymeleaf.prefix=classpath:/templates/
   spring.thymeleaf.suffix=.html
   spring.thymeleaf.cache=false
   ```

3. **Created Thymeleaf Templates**

   The .html template files were placed in the `src/main/resources/templates` directory.

4. **Implemented a Controller**

   A Spring MVC controller was created to render the Thymeleaf templates. The Model object was used to pass data to the view.